Ho Chi Minh City University of Technology (HCMUT), VNU-HCM

Faculty of Computer Science and Engineering

ASSIGNMENT 1

Foundations of Deep Learning Pipelines and Architectures

From Linear Models to Modern Sequence Models: A Comparative Study for Image Classification

Group G-03 | Nguyen Van An (2352013) | Vo Le Hai Dang (2352257) | Huynh Vuong Khang (2350011)

CO3133 - Deep Learning and Its Applications | Semester 261 | Instructor: Le Thanh Sach | October 2026

What this deck covers

Required area	Slides
Problem and data	3-5
Method	6-7
Experimental setup	8-9
Main results	10-12
Error analysis	13-14
Conclusion	15-16

All five models share one split, one preprocessing pipeline and one evaluation protocol.

Problem and data - the task

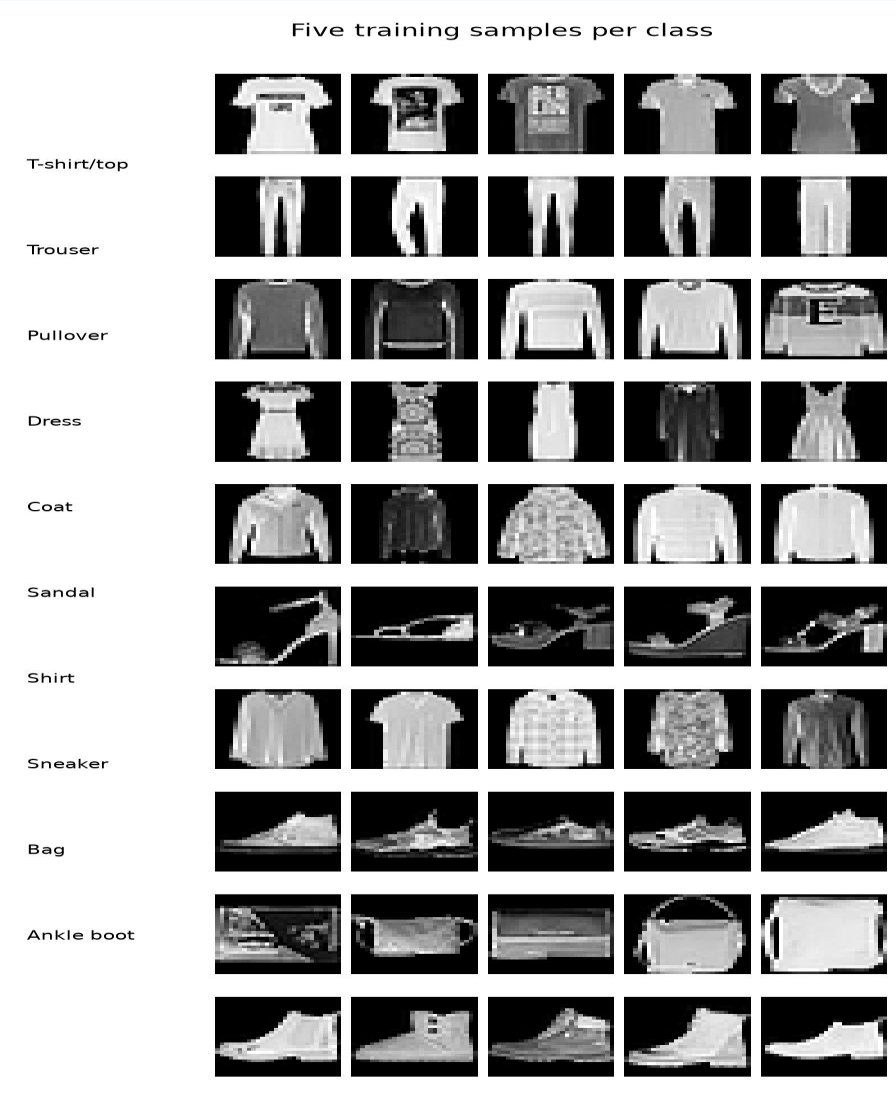
- Input: one 28x28 grayscale image, tensor (N, 1, 28, 28), uint8
- Output: one of ten clothing labels
- Prediction unit: one image; no part annotations
- Loss: cross-entropy on raw logits (no softmax before the loss)
- Optimiser: mini-batch Adam

Why it is hard

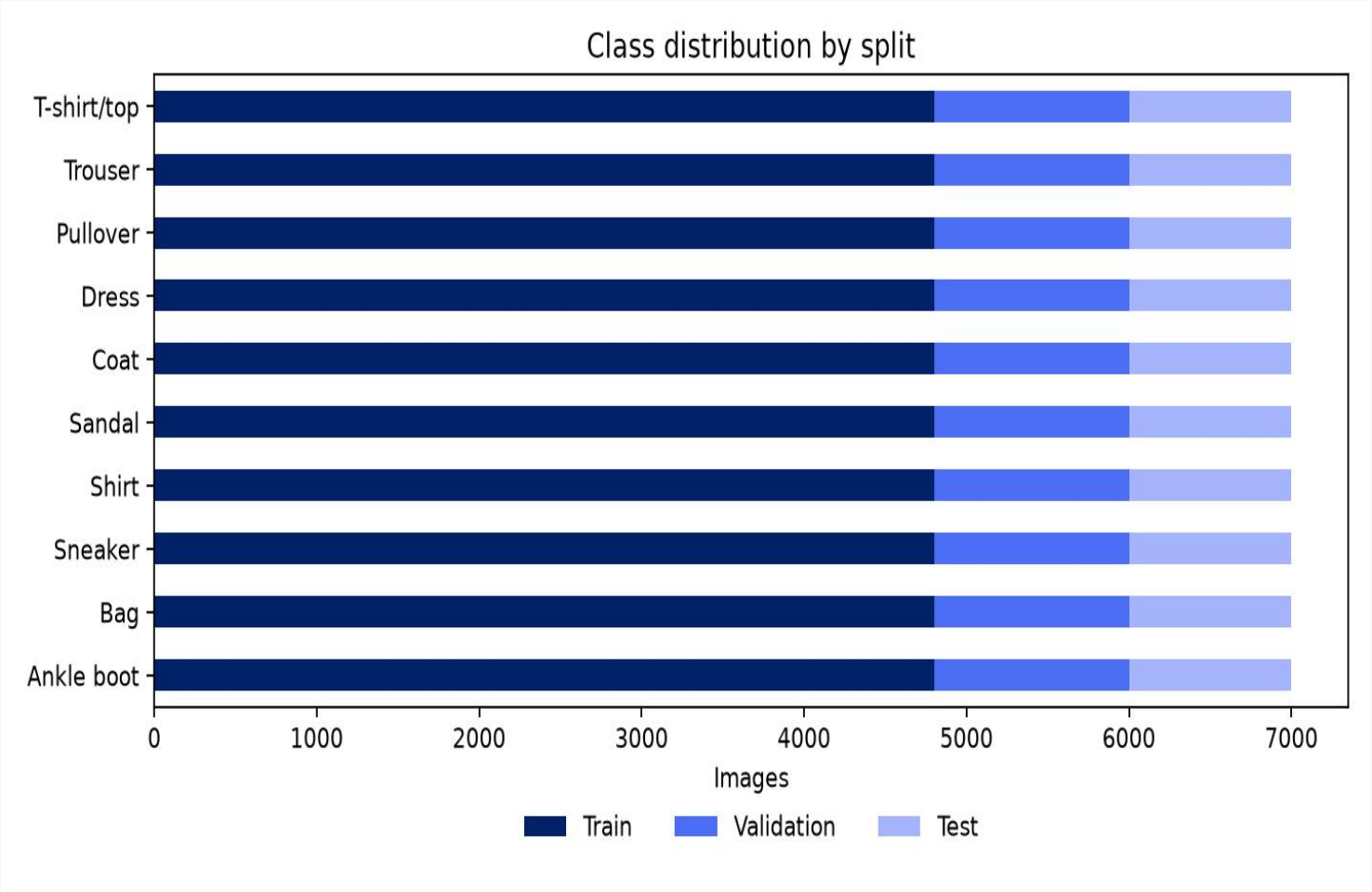
- **Silhouette ambiguity**: Shirt, T-shirt/top, Pullover, Coat look alike at 28x28
- **No colour:** grayscale removes the strongest remaining cue
- Sequence models must linearise a 2-D image into an order it does not have

Problem and data - dataset

Property	Value
Dataset	Fashion-MNIST (Zalando Research, 2017)
Source / access	torchvision.datasets.FashionMNIST, MD5-verified
License	MIT
Images	70,000 - exactly 7,000 per class
Format	28x28 grayscale; four gzip IDX files, ~30 MB
Split sizes	48,000 train / 12,000 validation / 10,000 test



Problem and data - dataset



Balance: every class contributes 4,800 / 1,200 / 1,000 images, so the maximum-to-minimum class ratio is 1.000. No class weighting or resampling is used.

Problem and data - split and preprocessing

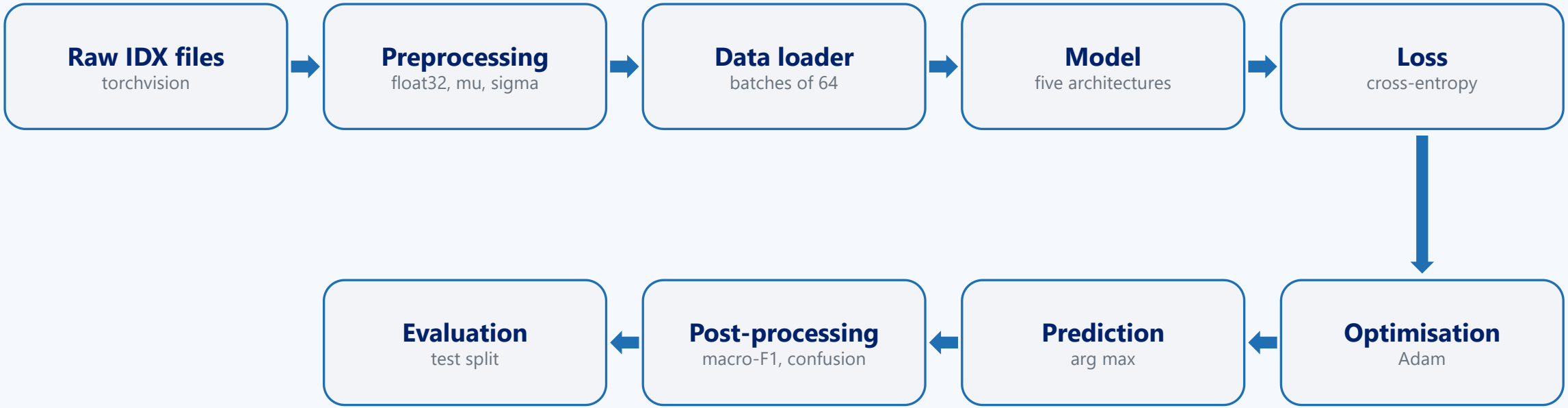
Split

- Official 60,000 / 10,000 train-test partition
- Validation carved from the training split only
- stratify=y, random_state=42, test_size=0.2
- Seed 42 shared by every model and every run
- Test split touched once per model, after checkpoint selection

Preprocessing

- uint8 -> float32, divided by 255
- Standardised with training-set mean 0.2863, std 0.3532
- Same statistics applied to validation and test (no leakage)
- No resizing, no cleaning needed; no augmentation in the main comparison
- Example batch: X (64,1,28,28) float32, y (64) int64

Method - the pipeline all five models share



One routine trains, validates, selects the checkpoint and evaluates all five families.

Method - the five architectures

Model	Input representation	Architecture	Params
Linear	flattened 784-vector	Flatten -> Linear(784, 10)	7,850
MLP	flattened 784-vector	784 -> 256 -> 128 -> 10, ReLU	235,146
CNN	28x28 image, kept 2-D	2 x [Conv 3x3 + ReLU + MaxPool 2x2], 32/64 ch; head 3136 -> 128 -> 256 -> 10	455,946
GRU	28 row tokens of 28 values	GRUCell loop, T=28, hidden 128; h_T -> Linear(128, 10)	61,962
Transformer	28 row tokens of 28 values	Linear(28->64), sinusoidal positions, 2 encoder layers (4 heads, FFN 128), mean pool -> Linear(64, 10)	69,450

- All five are self-implemented; only layers, optimiser, dataset access and metrics come from libraries (PyTorch, torchvision, scikit-learn, Matplotlib, pandas, NumPy)
- The CNN is self-designed and trained from scratch; no pretrained backbone or weights anywhere

Experimental setup - configuration

Item	Value
Optimiser / learning rate	Adam, 1e-3 constant (no scheduler)
Batch size / epochs	64 / 10 for every model
Loss	CrossEntropyLoss on raw logits
Gradient clipping	global norm 1.0, all models
Regularisation / early stopping	none (dropout 0.1 only inside the Transformer encoder) / none
Checkpoint criterion	highest validation macro-F1; that checkpoint is evaluated on test
Seed	42 (NumPy, Pytorch)
Hardware / software	RTX 3050 Laptop 4 GB, CUDA 13.0; Python 3.14.7, torch 2.14.0, torchvision 0.29.0
Determinism	Deterministic CUDA algorithms are not enabled, so same-seed re-runs may differ slightly. Determinism is still guaranteed when inferencing from a trained model checkpoint.

Experimental setup - design and decision rules

Hypotheses

- H1: a model with spatial inductive bias (CNN) beats models that flatten the image (Linear, MLP)
- H2: sequence models preserving one axis of order (GRU, Transformer) beat the flattening baselines without matching the CNN
- H3: accuracy is not monotone in parameter count

Controlled factors

- Changed: the architecture family only
- Fixed: dataset, split, preprocessing, batch size, epochs, optimiser, clipping, checkpoint rule, protocol

Metrics and decision rule

- Primary: test accuracy, test macro-F1
- Secondary: parameters, training time, inference time
- A gap counts only if > 1 point and consistent across both primary metrics

Main results - test split, selected checkpoints

Model	Params	Best ep	Val acc	Val F1	Test acc	Test F1	Train s	Infer ms
Linear	7,850	8	85.89%	85.94%	84.10%	84.15%	9.6	0.005
MLP	235,146	8	89.55%	89.57%	87.89%	87.89%	12.8	0.006
CNN	455,946	7	92.76%	92.77%	91.79%	91.79%	20.5	0.010
GRU	61,962	8	90.13%	90.09%	88.93%	88.85%	52.5	0.025
Transformer	69,450	8	89.60%	89.59%	88.71%	88.69%	32.5	0.012

The CNN is best on both primary metrics: 91.79% test accuracy and 91.79% test macro-F1. It leads the GRU by 2.86 points, the Transformer by 3.08, the MLP by 3.90 and the linear classifier by 7.69.

Accuracy and macro-F1 agree within 0.1 point for every model in this experiment. The balanced test split helps interpretation, but balance alone does not guarantee that the metrics will coincide. All reported numbers come from the checkpoint selected by validation macro-F1, reloaded before the test pass.

Main results - comparison and inductive bias

- H1 supported: the CNN beats both flattened baselines; Linear < MLP < CNN is consistent with increasing use of spatial structure. The CNN has the best F1 on all ten classes
- H2 supported: GRU 88.93% and Transformer 88.71% beat the MLP 87.89% using only row order, but neither approaches the CNN
- H3 supported: the MLP has 235,146 parameters at 87.89% while the GRU has 61,962 (about a quarter) at 88.93% - capacity is not the binding constraint, representation is

Trade-offs

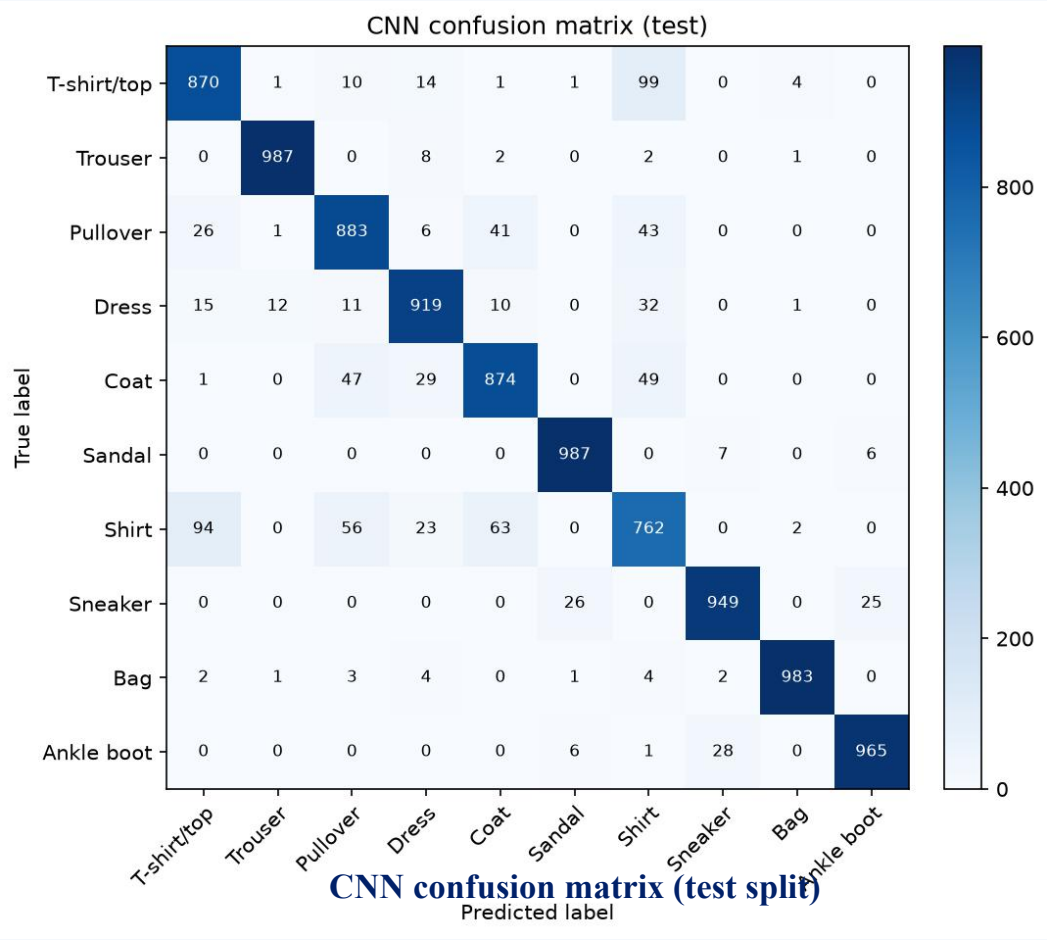
- GRU: most parameter-efficient, slowest to train (52.5 s, 4.1x the MLP) and slowest at inference (0.025 ms/sample, 2.5x the CNN) - 28 sequential cell steps cannot be parallelised
- Transformer: 1.6x faster to train and 2.1x faster at inference than the GRU for 0.22 points less accuracy
- Linear: 5.5x faster to train than the GRU and 58x fewer parameters than the CNN for 7.69 points less accuracy

Error analysis - where the errors are

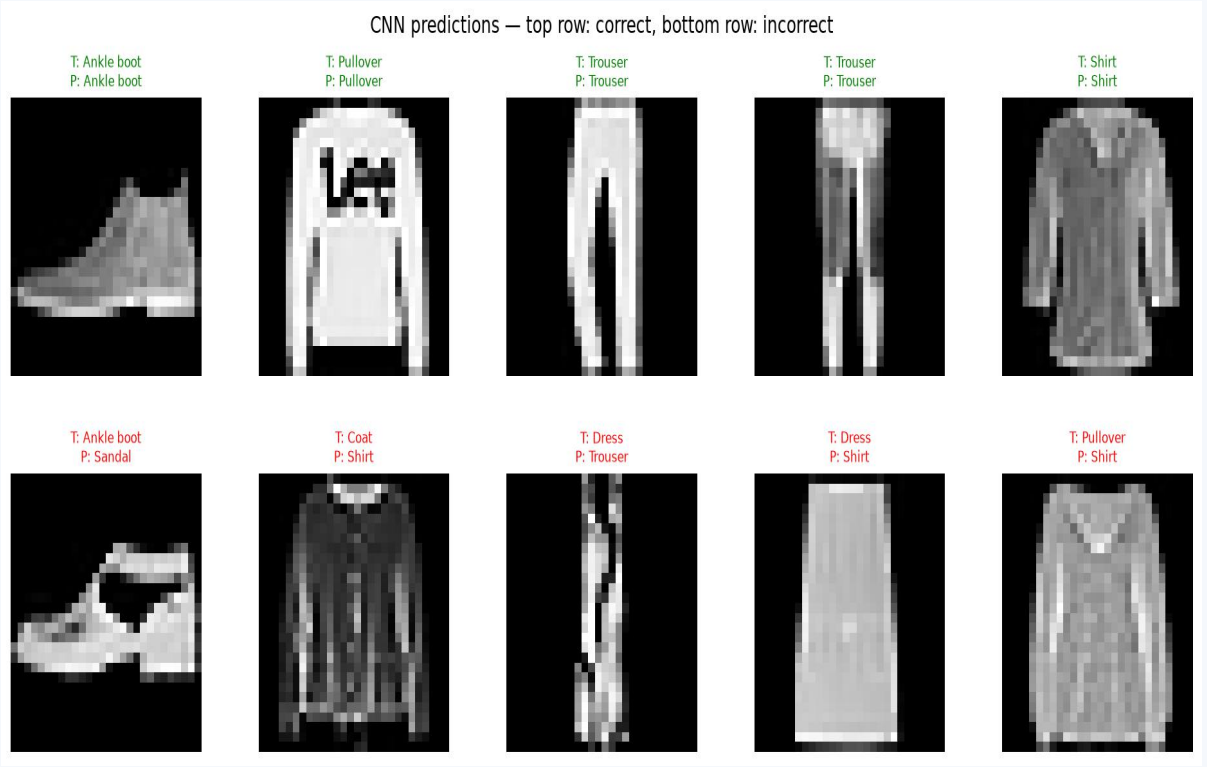
Error group	Linear	MLP	CNN	GRU	Transformer
Total errors (of 10,000)	1,590	1,211	821	1,107	1,129
Upper-body <-> upper-body	60.6%	64.7%	64.6%	62.0%	64.8%
Footwear <-> footwear	11.8%	10.2%	11.9%	10.7%	11.1%
Trouser <-> Dress	2.6%	2.1%	2.4%	1.7%	1.4%
Cross-group	25.0%	22.9%	21.1%	25.7%	22.7%

Group 1 - upper-body clothing (60-65% of errors): Shirt is the hardest class for all five models (F1 0.601 Linear, 0.682 MLP, 0.712 GRU, 0.710 Transformer, 0.765 CNN). Cause: a handful of pixels around shoulders and neckline decides the class, and grayscale removes fabric colour and texture.
Group 2 - footwear (10-12%): Sandal, Sneaker and Ankle boot share one footprint and differ in upper height and strap detail.
Group 3 - cross-group (21-26%): a long Coat or Pullover called a Dress, or a Bag called a T-shirt/top.

Error analysis - confusion matrix and examples



CNN confusion matrix (test split)



CNN predictions: top row correct, bottom row wrong

Largest CNN confusions: T-shirt/top -> Shirt 99, Shirt -> T-shirt/top 94, Shirt -> Coat 63, Shirt -> Pullover 56, Coat -> Shirt 49.

Conclusion

- Under one shared split, preprocessing and protocol, the self-designed CNN is the best of the five required families: 91.79% test accuracy and 91.79% test macro-F1
- It leads the GRU (88.93% / 88.85%), the Transformer (88.71% / 88.69%), the MLP (87.89%) and the linear classifier (84.10%)
- The ranking follows inductive bias rather than capacity: the most parameter-efficient model is not the largest, and flattening loses locality that dense capacity does not recover
- Spatial structure - not sequence order and not parameter count - is the dominant factor on this task
- The residual error is concentrated in one visually ambiguous class family, not spread across the label set

Limitations

- One dataset, grayscale, exactly balanced, single object per image
- One seed (42) with no variance estimate; fixed 10-epoch budget, no schedule, no early stopping; no architectural ablation; no augmentation in the main pipeline

Reproducibility and AI usage

Everything is traceable

- Config: assignment1/results/config.json
- Metrics: results/results.csv, results/results.json (per-class, confusion matrices)
- Histories: results/history/<model>.json
- Checkpoints: assignment1/checkpoints/*.pt
- Figures: results/curves, confusion, examples
- Rerun: assignment1/notebooks/main.ipynb with seed 42

AI usage

- Tools: ChatGPT-4 (page boilerplate), Gemini (EDA plotting, augmentation examples), opencode CLI agent (module refactoring, model documentation, notebook scaffolding)
- Typesetting of this deck and the report was AI-assisted
- Every number was produced by running assignment1/main.ipynb; artifacts were regenerated and histories cross-checked against the selected checkpoints
- Full log: assignment1/AI_USAGE.md